

CSA1715-ARTIFICIAL INTELLIGENCE

LAB PROGRAM- 9

30-07-2026

9. Write the python program to implement Travelling Salesman Problem.

Aim

To implement the Travelling Salesman Problem (TSP) using Python to find the minimum cost path that visits every city exactly once and returns to the starting city.

Algorithm

1. Start the program.
2. Define the cities and their distance matrix.
3. Select the first city as the starting city.
4. Generate all possible permutations of the remaining cities.
5. Add the starting city at the beginning and end of each route.
6. Calculate the total distance of each route.
7. Compare the calculated distance with the current minimum distance.
8. Store the route having the minimum distance.
9. Display the best route and minimum distance.
10. Create a weighted graph using NetworkX.
11. Display the distances between the cities.
12. Highlight the shortest route using Matplotlib.
13. Stop the program.

Program

```
import networkx as nx
import matplotlib.pyplot as plt
from itertools import permutations

d = [[0,10,15,20],
      [10,0,35,25],
      [15,35,0,30],
      [20,25,30,0]]

cities = ['A','B','C','D']
best = None
minimum = float('inf')

for p in permutations(cities[1:]):
    route = ['A'] + list(p) + ['A']
    distance = sum(d[cities.index(route[i])][cities.index(route[i+1])])
        for i in range(len(route)-1))

    if distance < minimum:
        minimum = distance
        best = route

print("Best Route:", " -> ".join(best))
print("Minimum Distance:", minimum)

G = nx.Graph()

for i in range(4):
    for j in range(i+1,4):
        G.add_edge(cities[i], cities[j], weight=d[i][j])

pos = {'A':(0,1), 'B':(2,2), 'C':(2,0), 'D':(4,1)}

nx.draw(G, pos, with_labels=True,
        node_size=1800, node_color="lightyellow")

nx.draw_networkx_edge_labels(
    G, pos, nx.get_edge_attributes(G,'weight'))

edges = [(best[i],best[i+1]) for i in range(len(best)-1)]
```

```
nx.draw_networkx_edges(G, pos, edgelist=edges, width=4)
```

```
plt.title("Travelling Salesman Problem")  
plt.show()
```

Input

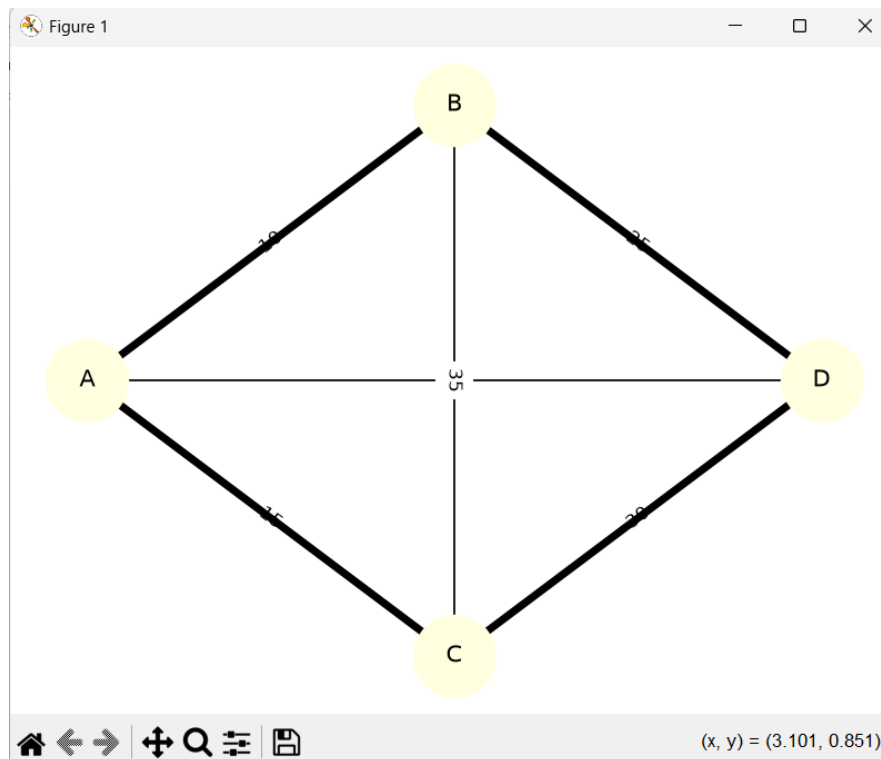
Distance Matrix:

	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

Output

Best Route: A -> B -> D -> C -> A

Minimum Distance: 80



Result:

The Travelling Salesman Problem was successfully implemented using Permutation Search. The program found the optimal route with the minimum travelling cost.